Documentación técnica

Sistema experto con interfaz web para diagnosticar problemas de rendimiento en videojuegos de PC

Inteligencia Artificial II — Proyecto final · Fundación Universitaria Los Libertadores
Juan F. Vanegas Silva · Diego Felipe Almanza Vanegas · Noviembre de 2026

1. Introducción

1.1 Descripción del dominio

Cuando un videojuego "va mal" en el PC, el jugador solo ve el síntoma: pocos FPS, tirones (*stuttering*) o lag. La causa real puede ser muy distinta —la tarjeta gráfica al límite, el procesador saturado, sobrecalentamiento, falta de RAM, el disco duro o la red— y cada una se soluciona de forma diferente. Comprar una GPU nueva no arregla un problema de temperatura, y bajar los gráficos no arregla el lag de una conexión WiFi.

El sistema captura cómo razona un jugador con experiencia en hardware: lee las lecturas de monitoreo (FPS, uso de GPU, CPU y RAM, temperatura, ping) y los síntomas observados, deduce la causa probable con un nivel de certeza, recomienda la acción concreta y explica su razonamiento.

1.2 Justificación y relevancia

- Es un problema real y frecuente para cualquier persona que juega en PC, y las lecturas se obtienen gratis con herramientas como MSI Afterburner o el administrador de tareas.
- El conocimiento se expresa naturalmente como reglas SI-ENTONCES ("si la GPU está al 99 % y hay pocos FPS, la GPU es el límite").
- El diagnóstico no es 100 % seguro (un síntoma puede tener varias causas), lo que justifica usar factores de certeza.
- Es la continuación de la Actividad 4: se expandió de 12 a 15 reglas y se resolvieron las limitaciones que se identificaron allí (hechos binarios, sin negación, sin combinación de evidencia).

1.3 Alcance

Cubre	No cubre
Cuello de botella de GPU, CPU y disco	Fallas físicas (pantallazos azules, artefactos)
Sobrecalentamiento (*thermal throttling*)	Drivers, sistema operativo y bugs propios del juego
Falta de memoria RAM	Fuente de poder, resolución de pantalla
Lag por WiFi y lag del servidor o proveedor de internet	Diagnóstico detallado del router o del proveedor; consolas y celulares

2. Diseño del sistema

2.1 Arquitectura general

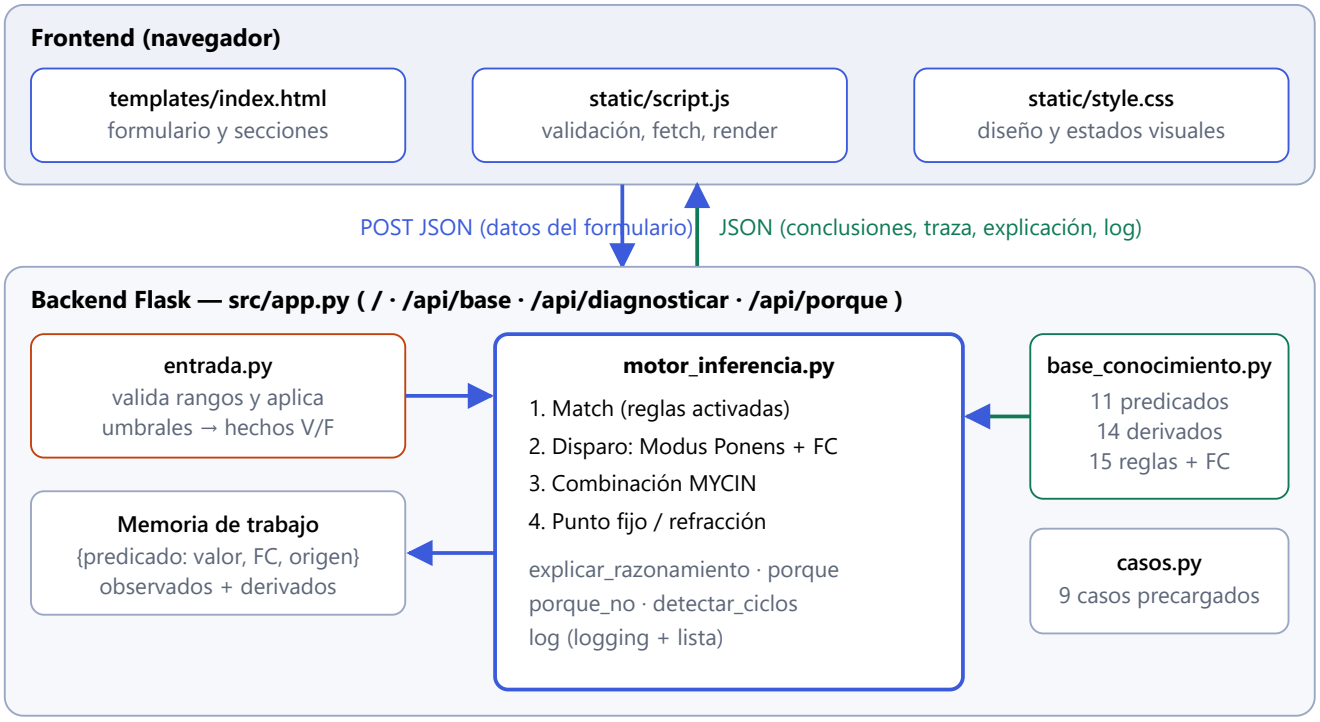


Figura 1. Arquitectura: el frontend solo envía datos y dibuja; toda la lógica del SBC vive en el backend.

2.2 Componentes principales

Componente	Archivo	Responsabilidad
Base de conocimiento	`src/base_conocimiento.py`	Predicados (con tipo, dominio, rango y umbral), textos de conclusiones y las 15 reglas con FC, fórmula lógica y justificación. Solo datos, sin lógica.
Capa de entrada	`src/entrada.py`	Valida el formulario (tipos, rangos, opciones) y convierte lecturas numéricas en hechos verdaderos o falsos aplicando el umbral de cada predicado.
Motor de inferencia	`src/motor_inferencia.py`	Encadenamiento hacia adelante, FC, combinación MYCIN, explicación (`explicar_razonamiento` , `porque` , `porque_no`), detección de ciclos y log.
Casos de prueba	`src/casos.py`	9 casos con datos y resultado esperado; los usan las pruebas, el notebook y la interfaz (casos precargados).
Aplicación web	`src/app.py`	Servidor Flask: sirve la página y expone la API JSON; traduce errores de validación a HTTP 400.
Interfaz	`templates/, static/`	Formulario con medidores de umbral y vista previa del predicado, validación en el navegador, **mapa de inferencia** (SVG generado desde `/api/base`), tarjetas con anillos de certeza, árbol "¿Por qué?", "¿Por qué no?", cadena de reglas, log, pestaña de la base y botón de efectos.

2.3 Flujo de datos

1. El usuario llena el formulario (o carga un caso) y presiona **Diagnosticar**. `script.js` valida los rangos y envía `POST /api/diagnosticar` con un JSON como `{"fps": 20, "uso_cpu": 97, "tirones": "si", ...}` .

2. `app.py` llama a `hechos_desde_formulario()`: si hay errores responde 400 con la lista de mensajes; si no, obtiene los hechos iniciales, por ejemplo `{FPSBajos: True, UsoCPUAlto: True, ...}`.
3. `diagnosticar()` ejecuta el motor sobre la base de conocimiento y arma la respuesta: diagnósticos y recomendaciones ordenados por certeza, reglas aplicadas en orden, explicación en texto, árbol de justificación de cada conclusión, conclusiones no alcanzadas y log.
4. `script.js` dibuja la respuesta. El botón **¿Por qué?** despliega el árbol ya calculado; **¿Por qué no...?** llama a `POST /api/porque`.

Método	Ruta	Entrada	Salida
GET	`/`	—	Página HTML
GET	`/api/base`	—	Predicados, reglas (lógica y SI-ENTONCES), casos precargados, ciclos detectados
POST	`/api/diagnosticar`	Datos del formulario	Diagnósticos, recomendaciones, reglas aplicadas, explicación, árboles, log · 400 si los datos no son válidos
POST	`/api/porque`	`{datos, conclusion}`	Árbol de justificación o, si no se concluyó, estado de cada condición de las reglas que la concluirían

3. Base de conocimiento

3.1 Predicados formalmente definidos

Sea **E** el conjunto de equipos (PC) en una sesión de juego. Todos los predicados son de aridad 1, **P : E → {V, F}**, con x ∈ E. Los 11 predicados **observables** son los hechos de entrada; los ingresa el usuario directamente o se calculan desde una lectura numérica con un umbral:

Predicado	Tipo	Dominio → Rango	Significado	Se calcula con
`FPSBajos(x)`	Observable	E → {V, F}	El juego corre a menos de 30 FPS	`fps < 30`
`UsoGPUAlto(x)`	Observable	E → {V, F}	La GPU está al 95 % de uso o más mientras se juega	`uso_gpu ≥ 95 %`
`UsoCPUAlto(x)`	Observable	E → {V, F}	La CPU está al 90 % de uso o más mientras se juega	`uso_cpu ≥ 90 %`
`GraficosAltos(x)`	Observable	E → {V, F}	El juego está configurado en calidad alta o ultra	`calidad ∈ {alta, ultra}`
`TempAlta(x)`	Observable	E → {V, F}	La CPU o la GPU supera los 90 °C	`temperatura > 90 °C`
`FPSCaenConTiempo(x)`	Observable	E → {V, F}	Los FPS empiezan bien y bajan después de varios minutos de juego	`respuesta = sí`
`Tirones(x)`	Observable	E → {V, F}	Hay congelamientos breves (stuttering) durante el juego	`respuesta = sí`
`RAMLlena(x)`	Observable	E → {V, F}	El uso de memoria RAM es del 90 % o más	`uso_ram ≥ 90 %`
`JuegoEnHDD(x)`	Observable	E → {V, F}	El juego está instalado en un disco duro mecánico (no SSD)	`almacenamiento = HDD`
`PingAlto(x)`	Observable	E → {V, F}	La latencia en juegos en línea supera los 100 ms	`ping > 100 ms`
`ConexionWiFi(x)`	Observable	E → {V, F}	El equipo se conecta a internet por WiFi	`conexión = WiFi`

Cada hecho puede quedar en **tres estados**: verdadero, falso (el usuario lo afirmó) o desconocido (campo vacío o "No sé"). Un hecho desconocido no satisface ni P(x) ni ¬P(x).

Los 14 predicados **derivados** (mismo dominio y rango) los concluye el motor:

Predicado derivado	Tipo	Texto que ve el usuario
`CuelloBotellaGPU(x)`	Diagnóstico	La tarjeta gráfica (GPU) limita el rendimiento
`CuelloBotellaCPU(x)`	Diagnóstico	El procesador (CPU) limita el rendimiento
`ThermalThrottling(x)`	Diagnóstico	Sobrecalentamiento (thermal throttling)
`FaltaRAM(x)`	Diagnóstico	La memoria RAM no es suficiente
`CuelloBotellaDisco(x)`	Diagnóstico	El disco duro no carga los datos a tiempo
`ProblemaRedLocal(x)`	Diagnóstico	El lag viene de la conexión WiFi
`ProblemaServidor(x)`	Diagnóstico	El lag viene del servidor o del proveedor de internet
`RecBajarGraficos(x)`	Recomendación	Bajar la calidad gráfica o activar DLSS/FSR

`RecCerrarProcesos(x)`	Recomendación	Cerrar programas en segundo plano
`RecMantenimientoTermico(x)`	Recomendación	Hacer mantenimiento térmico
`RecAmpliarRAM(x)`	Recomendación	Ampliar la memoria RAM
`RecMoverASSD(x)`	Recomendación	Mover el juego a un SSD
`RecCableEthernet(x)`	Recomendación	Conectar el PC por cable Ethernet
`RecCambiarServidor(x)`	Recomendación	Elegir un servidor más cercano o revisar el proveedor

3.2 Reglas en lógica de predicados y en formato SI-ENTONCES, con su justificación

Todas las reglas están cuantificadas universalmente (∀x): valen para cualquier equipo. La traducción a reglas de producción elimina el ∀x; el motor instancia x con el equipo consultado (**instanciación universal**) y aplica **Modus Ponens**. El cuantificador existencial aparece en las consultas: `porque()` responde preguntas como ∃x ThermalThrottling(x). Las reglas R1–R12 vienen de la Actividad 4; R13–R15 son nuevas.

Regla	Grupo	Lógica de predicados	SI – ENTONCES	FC	Justificación
R1	Hardware	`∀x (FPSBajos(x) ∧ UsoGPUAlto(x) → CuelloBotellaGPU(x))`	`SI FPSBajos Y UsoGPUAlto ENTONCES CuelloBotellaGPU`	0.85	Si la GPU ya está al máximo y no alcanza 30 FPS, ella es el límite.
R2	Hardware	`∀x (FPSBajos(x) ∧ UsoCPUAlto(x) → CuelloBotellaCPU(x))`	`SI FPSBajos Y UsoCPUAlto ENTONCES CuelloBotellaCPU`	0.80	Con el procesador saturado, la GPU espera los datos de cada cuadro.
R3	Hardware	`∀x (CuelloBotellaGPU(x) ∧ GraficosAltos(x) → RecBajarGraficos(x))`	`SI CuelloBotellaGPU Y GraficosAltos ENTONCES RecBajarGraficos`	0.90	Bajar la calidad gráfica le quita carga directamente a la GPU.
R4	Hardware	`∀x (CuelloBotellaCPU(x) → RecCerrarProcesos(x))`	`SI CuelloBotellaCPU ENTONCES RecCerrarProcesos`	0.70	Los programas en segundo plano le roban tiempo de CPU al juego.
R5	Temperatura	`∀x (TempAlta(x) ∧ FPSCaenConTiempo(x) → ThermalThrottling(x))`	`SI TempAlta Y FPSCaenConTiempo ENTONCES ThermalThrottling`	0.85	El calor acumulado obliga al hardware a bajar su frecuencia.
R6	Temperatura	`∀x (ThermalThrottling(x) → RecMantenimientoTermico(x))`	`SI ThermalThrottling ENTONCES RecMantenimientoTermico`	0.90	Limpiar ventiladores y cambiar la pasta térmica baja la temperatura.
R7	Memoria y disco	`∀x (Tirones(x) ∧ RAMLlena(x) → FaltaRAM(x))`	`SI Tirones Y RAMLlena ENTONCES FaltaRAM`	0.80	Sin RAM libre el sistema usa el archivo de paginación y se congela.
R8	Memoria y disco	`∀x (FaltaRAM(x) → RecAmpliarRAM(x))`	`SI FaltaRAM ENTONCES RecAmpliarRAM`	0.75	Más memoria evita el uso del disco como memoria virtual.

R9	Memoria y disco	`∀x (Tirones(x) ∧ JuegoEnHDD(x) → CuelloBotellaDisco(x))`	`SI Tirones Y JuegoEnHDD ENTONCES CuelloBotellaDisco`	0.70	Un HDD es lento para cargar datos sobre la marcha (FC bajo: los tirones también pueden venir de la compilación de shaders).
R10	Memoria y disco	`∀x (CuelloBotellaDisco(x) → RecMoverASSD(x))`	`SI CuelloBotellaDisco ENTONCES RecMoverASSD`	0.85	Un SSD carga los datos decenas de veces más rápido que un HDD.
R11	Red	`∀x (PingAlto(x) ∧ ConexionWiFi(x) → ProblemaRedLocal(x))`	`SI PingAlto Y ConexionWiFi ENTONCES ProblemaRedLocal`	0.75	El WiFi agrega latencia e interferencia frente al cable.
R12	Red	`∀x (ProblemaRedLocal(x) → RecCableEthernet(x))`	`SI ProblemaRedLocal ENTONCES RecCableEthernet`	0.85	El cable da una conexión estable y con menos latencia.
R13	Memoria y disco	`∀x (FaltaRAM(x) → RecCerrarProcesos(x))`	`SI FaltaRAM ENTONCES RecCerrarProcesos`	0.60	Cerrar programas libera memoria. Comparte conclusión con R4: si ambas se disparan, el motor combina sus certezas (fórmula de MYCIN).
R14	Red	`∀x (PingAlto(x) ∧ ¬ConexionWiFi(x) → ProblemaServidor(x))`	`SI PingAlto Y NO ConexionWiFi ENTONCES ProblemaServidor`	0.60	Descartada la red inalámbrica, la latencia se origina fuera del equipo (FC bajo: también podría ser el router).
R15	Red	`∀x (ProblemaServidor(x) → RecCambiarServidor(x))`	`SI ProblemaServidor ENTONCES RecCambiarServidor`	0.70	Un servidor más cercano reduce la distancia que recorren los paquetes.

3.3 Factores de certeza

- Hechos observados: FC = 1.0.
- Al aplicar una regla: **FC(conclusión) = FC(regla) × min(FC(condiciones))**; una conjunción es tan confiable como su condición más débil.
- Si dos reglas concluyen lo mismo (R4 y R13 → RecCerrarProcesos), se combinan como evidencias independientes (MYCIN): **$FC = FC_1 + FC_2 \cdot (1 - FC_1)$**. Ejemplo del caso 5: 0.56 + 0.48·(1 − 0.56) = 0.77.
- La certeza baja a medida que la cadena crece: R1 da CuelloBotellaGPU con 0.85 y R3 da RecBajarGraficos con 0.90 × 0.85 = 0.77.
- Los FC más bajos reflejan causas alternativas: R9 (0.70) porque los tirones con HDD también pueden venir de la compilación de *shaders*; R14 (0.60) porque el ping alto por cable también puede ser el router.

4. Motor de inferencia

4.1 Algoritmo de encadenamiento hacia adelante

El motor implementa el ciclo de reconocimiento-acción guiado por los datos: parte de los hechos y aplica reglas hasta que no se pueda deducir nada nuevo (punto fijo). La resolución de conflictos es simple y determinista: se disparan **todas** las reglas activadas en el orden de la base, calculando su FC con la memoria del inicio de la iteración. Así cada iteración corresponde a un nivel de la cadena (iteración 1 = diagnósticos, iteración 2 = recomendaciones, iteración 3 = punto fijo).

```
encadenamiento_adelante(base, hechos_iniciales):
    memoria  ← { p: (valor, FC = 1.0, "observado") para cada hecho inicial }
    disparadas ← ∅ ;  iteracion ← 0
    mientras iteracion < MAX_ITERACIONES:
        iteracion ← iteracion + 1
        agenda ← [ r ∈ reglas  |  r ∉ disparadas  y  toda condición (P, v) de r cumple memoria[P].valor
= v ]
        si agenda está vacía:  PUNTO FIJO → salir
        para cada r en agenda (orden de la base, FC calculado con la memoria del INICIO de la
iteración):
            disparadas ← disparadas ∪ {r}                      # refracción
            fc ← FC(r) × min(FC de sus condiciones)             # Modus Ponens con certeza
            si conclusión(r) no está en memoria:  memoria[c] ← (V, fc, [r])
            si no:  memoria[c].FC ← FC_previa + fc·(1 − FC_previa)   # combinación MYCIN
            registrar paso en reglas_aplicadas y en el log
    devolver memoria, reglas_aplicadas
```

4.2 Mecanismo de explicación

- `explicar_razonamiento(reglas_aplicadas)`: la cadena completa, en orden, con iteración, regla, condiciones, conclusión y cálculo del FC (incluida la combinación).
- `porque(conclusion, hechos_iniciales, reglas_aplicadas)`: recorre hacia atrás desde la conclusión hasta los hechos observados (árbol de justificación). Si varias reglas soportan la conclusión, muestra cada rama. En la web es el botón **¿Por qué?**.
- `porque_no(conclusion, hechos_finales, base)`: para cada regla que concluiría el predicado, indica si cada condición se cumple, es falsa, es un dato no informado o es un derivado no concluido. Le dice al usuario qué dato le falta aportar.
- **Log**: cada paso se registra con el módulo `logging` (logger `sbc.motor`, visible en la consola del servidor) y en una lista que se muestra en la interfaz.

4.3 Manejo de casos especiales

Situación	Comportamiento
No se puede derivar nada	El motor alcanza el punto fijo en la iteración 1; la respuesta trae `sin_conclusion = true` y la interfaz muestra un aviso con sugerencia de usar "¿Por qué no...?".
Ciclos infinitos	Dos niveles: (1) **refracción**, cada regla se dispara una vez, así que el motor termina en ≤ |R| + 1 iteraciones; tope `MAX_ITERACIONES = 50` con aviso en el log. (2) `detectar_ciclos()` revisa el grafo de dependencias con DFS al iniciar la aplicación; la pestaña "Base de conocimiento" muestra el resultado.
Negación y datos desconocidos	¬P(x) solo se cumple si el usuario afirmó que P es falso. Ping alto con conexión desconocida no dispara ni R11 ni R14.

Dos reglas, misma conclusión	Se combinan los FC con MYCIN y la conclusión registra ambas reglas como origen.
Regla que contradice una observación	Se conserva el hecho observado y se registra una advertencia en el log.
Entrada inválida	Valores fuera de rango, no numéricos, opciones desconocidas o formulario vacío: HTTP 400 con mensajes claros, validados también en el navegador.

5. Implementación técnica

5.1 Tecnologías utilizadas

- **Python 3.8+** (probado en 3.13): el motor y la base de conocimiento solo usan la biblioteca estándar (`logging`, `dataclasses`).
- **Flask 3**: servidor web y API JSON.
- **HTML, CSS y JavaScript** sin frameworks ni librerías de JavaScript. Las tipografías (Russo One y Chakra Petch) se piden a Google Fonts solo si hay internet; sin conexión se usan Bahnschrift o Segoe UI de Windows, así que la aplicación funciona offline.
- **CSS moderno para animaciones**: View Transitions (cambio de pestaña), `@property` (anillos de certeza y bordes giratorios), animaciones guiadas por scroll (filas de la base), `interpolate-size` (árbol desplegable) y `@starting-style` (avisos). Todo es mejora progresiva: si el navegador no lo soporta, el contenido se muestra igual.
- **unittest** para pruebas automáticas (incluye pruebas de la API con el cliente de pruebas de Flask) y **Jupyter** para el notebook de validación.

5.2 Decisiones de diseño importantes

- **Flask en lugar de Streamlit:** separa claramente backend (API) y frontend, permite explicar la integración y da control total del diseño (árbol desplegable, estados de carga, errores).
- **La interfaz no reimplementa lógica:** solo envía datos y dibuja la respuesta. El notebook, las pruebas y la web usan los mismos módulos y dan los mismos resultados.
- **Formato de reglas del enunciado** `(predicado, valor)`: el valor `False` permite expresar la negación sin cambiar la estructura.
- **Lecturas numéricas + umbrales**: el usuario no tiene que decidir qué es "alto"; los umbrales están en un solo lugar (`entrada.py`) y documentados en `PREDICADOS`.
- **Tres estados en los hechos**: "No sé" no es "No". Evita conclusiones falsas por asumir el mundo cerrado.
- **Disparo por niveles** (memoria del inicio de la iteración): hace la traza fácil de leer y explicar.
- **Recomendaciones ordenadas por certeza**: lo primero que ve el usuario es lo más probable que funcione.
- **Animaciones con significado y accesibles**: el mapa se ilumina por iteración (muestra el encadenamiento), solo se animan `transform` y `opacity`, y un botón "Efectos" permite apagarlas; por defecto sigue `prefers-reduced-motion` del sistema.

5.3 Desafíos encontrados y soluciones

Desafío	Solución
Con negación por mundo cerrado, "ping alto" sin informar la conexión culpaba al servidor.	Hechos de tres estados y negación explícita.

Dos reglas para la misma recomendación: la segunda se ignoraba.	Combinación MYCIN y registro de ambas reglas como origen; el árbol "¿Por qué?" muestra las dos ramas.
Valores en el umbral (30 FPS, GPU 95 %) generaban dudas.	Umbrales explícitos (< 30, ≥ 95) documentados y probados (caso 9).
Flask ordenaba alfabéticamente las claves del JSON y desordenaba las tablas.	`app.json.sort_keys = False`.
Mantener sincronizados notebook, pruebas y casos precargados.	Un único archivo `casos.py` usado por los tres.
En el equipo de pruebas Windows tenía las animaciones desactivadas, así que la interfaz respetaba esa preferencia y no animaba nada.	Atributo `data-motion` en la raíz: por defecto sigue la preferencia del sistema y el botón "Efectos" lo cambia y lo recuerda.
El mapa de inferencia debía reflejar la base sin duplicarla.	`/api/base` expone las condiciones y la conclusión de cada regla; el SVG se genera a partir de ellas, así que una regla nueva aparece sola en el mapa.

6. Validación

6.1 Tabla de casos de prueba

Las reglas y conclusiones esperadas se definieron antes de ejecutar el sistema. La columna "Resultado obtenido" se generó ejecutando el motor; `python -m unittest discover -s tests -v` verifica lo mismo (14 pruebas, todas correctas).

Caso	Descripción	Hechos iniciales	Reglas esperadas	Conclusión esperada (FC)	Resultado obtenido	Estado
1	**PC con juego en ultra**. Juego exigente en ultra; la GPU está al 99 % y no pasa de 25 FPS.	FPSBajos, UsoGPUAlto, GraficosAltos	R1, R3	CuelloBotellaGPU (0.85) RecBajarGraficos (0.77)	CuelloBotellaGPU (0.85) RecBajarGraficos (0.77) *R1, R3 · 3 iter.*	✓
2	**Portátil caliente**. Portátil que llega a 96 °C, pierde FPS con el tiempo y se congela con la RAM llena.	FPSBajos, TempAlta, RAMLlena, FPSCaenConTiempo, Tirones	R5, R7, R6, R8, R13	ThermalThrottling (0.85) FaltaRAM (0.80) RecMantenimientoTermico (0.77) RecAmpliarRAM (0.60) RecCerrarProcesos (0.48)	ThermalThrottling (0.85) FaltaRAM (0.80) RecMantenimientoTermico (0.77) RecAmpliarRAM (0.60) RecCerrarProcesos (0.48) *R5, R7, R6, R8, R13 · 3 iter.*	✓
3	**Lag por WiFi**. Partida en línea con 180 ms de ping conectado por WiFi.	PingAlto, ConexionWiFi	R11, R12	ProblemaRedLocal (0.75) RecCableEthernet (0.64)	ProblemaRedLocal (0.75) RecCableEthernet (0.64) *R11, R12 · 3 iter.*	✓
4	**Lag con cable (negación)**. Ping de 150 ms aunque el PC está por cable: la red local queda descartada.	PingAlto, ¬ConexionWiFi	R14, R15	ProblemaServidor (0.60) RecCambiarServidor (0.42)	ProblemaServidor (0.60) RecCambiarServidor (0.42) *R14, R15 · 3 iter.*	✓

5	**PC viejo (combinación de FC)**. CPU saturada, poca RAM y juego en HDD: dos reglas recomiendan cerrar procesos.	FPSBajos, UsoCPUAlto, RAMLlena, JuegoEnHDD, Tirones	R2, R7, R9, R4, R8, R10, R13	CuelloBotellaCPU (0.80) FaltaRAM (0.80) CuelloBotellaDisco (0.70) RecCerrarProcesos (0.77) RecAmpliarRAM (0.60) RecMoverASSD (0.59)	CuelloBotellaCPU (0.80) FaltaRAM (0.80) CuelloBotellaDisco (0.70) RecCerrarProcesos (0.77) RecAmpliarRAM (0.60) RecMoverASSD (0.59) *R2, R7, R9, R4, R8, R10, R13 · 3 iter.*	✓
6	**Sin conclusión**. Gráficos en ultra pero el juego va a 90 FPS: no hay ningún problema que diagnosticar.	¬FPSBajos, ¬UsoGPUAlto, GraficosAltos	Ninguna	Ninguna	Ninguna *sin reglas · 1 iter.*	✓
7	**Todo falla (máxima cobertura)**. Equipo con todos los síntomas a la vez: activa 13 de las 15 reglas.	FPSBajos, UsoGPUAlto, UsoCPUAlto, TempAlta, RAMLlena, PingAlto, GraficosAltos, JuegoEnHDD, ConexionWiFi, FPSCaenConTiempo, Tirones	R1, R2, R5, R7, R9, R11, R3, R4, R6, R8, R10, R12, R13	CuelloBotellaGPU (0.85) CuelloBotellaCPU (0.80) ThermalThrottling (0.85) FaltaRAM (0.80) CuelloBotellaDisco (0.70) ProblemaRedLocal (0.75) RecBajarGraficos (0.77) RecCerrarProcesos (0.77) RecMantenimientoTermico (0.77) RecAmpliarRAM (0.60) RecMoverASSD (0.59) RecCableEthernet (0.64)	CuelloBotellaGPU (0.85) CuelloBotellaCPU (0.80) ThermalThrottling (0.85) FaltaRAM (0.80) CuelloBotellaDisco (0.70) ProblemaRedLocal (0.75) RecBajarGraficos (0.77) RecCerrarProcesos (0.77) RecMantenimientoTermico (0.77) RecAmpliarRAM (0.60) RecMoverASSD (0.59) RecCableEthernet (0.64) *R1, R2, R5, R7, R9, R11, R3, R4, R6, R8, R10, R12, R13 · 3 iter.*	✓
8	**Borde: conexión desconocida**. Ping alto pero el usuario no sabe cómo se conecta: no se puede culpar ni al WiFi ni al servidor.	PingAlto	Ninguna	Ninguna	Ninguna *sin reglas · 1 iter.*	✓

9	**Borde: valores en el umbral**. 30 FPS exactos (el umbral es < 30) con GPU al 95 % y CPU al 89 %: FPSBajos es falso.	¬FPSBajos, UsoGPUAlto, ¬UsoCPUAlto, GraficosAltos	Ninguna	Ninguna	Ninguna *sin reglas · 1 iter.*	✓

6.2 Análisis de resultados

- **9/9 casos correctos**: reglas, orden y FC coinciden con lo esperado. Entre todos se ejercitan las **15 reglas**.
- Caso 7 activa **13 de 15 reglas**; R14 y R15 no pueden dispararse a la vez que R11 y R12 porque exigen ¬ConexionWiFi.
- Caso 5 muestra la **combinación de evidencia**: cerrar procesos pasa de 0.56 a 0.77 y queda como la recomendación principal.
- Casos 6, 8 y 9 muestran que el sistema **no inventa diagnósticos**: sin datos suficientes, con un dato desconocido o justo en el umbral, no concluye nada y lo explica.
- Pruebas adicionales: entradas inválidas (HTTP 400), base con ciclo (detectado y el motor termina), `porque_no()` y API.

6.3 Limitaciones identificadas

- Los FC se estimaron con nuestra experiencia como jugadores, no con datos medidos.
- La combinación MYCIN ocurre en una conclusión hoja; si se combinara un diagnóstico intermedio, las recomendaciones ya derivadas de él no se recalcularían.
- Umbrales nítidos: 89 % de CPU no es "alto" y 90 % sí (no hay lógica difusa).
- Una sola máquina por consulta y sin historial.

7. Conclusiones

7.1 Logros alcanzados

- SBC completo con 15 reglas formalizadas en lógica de predicados, motor de encadenamiento hacia adelante con FC, combinación de evidencia, negación y detección de ciclos.
- Explicación en tres niveles: cadena completa, "¿Por qué?" y "¿Por qué no?".
- Aplicación web funcional, con validación, casos precargados y manejo de errores, integrada con el motor por una API JSON.

7.2 Aprendizajes obtenidos

Separar conocimiento (datos), inferencia (algoritmo) e interfaz hizo posible probar cada parte por separado y agregar reglas sin tocar la web. Formalizar en lógica mostró qué condiciones eran realmente necesarias, y la negación nos enseñó la diferencia entre "falso" y "desconocido".

7.3 Posibles mejoras futuras

- Lógica difusa para los umbrales y FC calibrados con mediciones reales.

- Modo entrevista con encadenamiento hacia atrás que pregunte solo los datos que faltan (usando `porque_no`).
- Más reglas: drivers, fuente de poder, resolución; importar el log de MSI Afterburner.